



SECP
3133
Section 2

Optimizing High- Performance Data Processing For Large-Scale Web Crawlers

PRESENTED BY

Group ABC

GROUP MEMBER

JOANNE CHING YIN XUAN (A23CS0227)

LIM YU HAN (A23CS0241)

CHUA JIA LIN (A23CS0069)

EVELYN GOH YUAN QI (A23CS0222)



An abstract graphic on the left side of the slide featuring a dark blue background with a network of glowing blue nodes and connecting lines, resembling a digital or data network.

Project Background

Problem:

- News websites publish thousands of articles daily
- Manual data collection is slow and inefficient
- Large datasets require efficient crawling and processing

Proposed Solution:

Develop a **high-performance NST web crawler** capable of collecting over 100,000 article metadata records with optimized data processing

Objectives

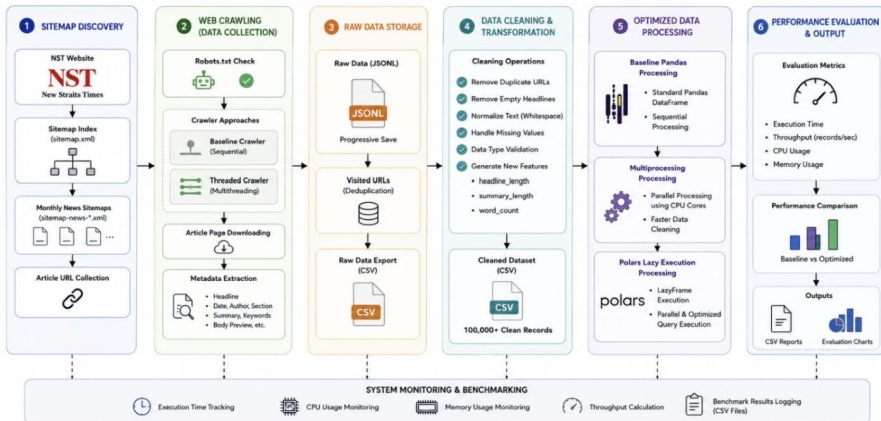
- » Develop an automated web crawler for NST
- » Collect more than 100,000 news records
- » Store data in JSONL and CSV formats
- » Improve performance using:
 - Multithreading
 - Multiprocessing
 - Polars Lazy Execution
- » Evaluate execution time, CPU usage, memory usage and throughput



System Architecture

SYSTEM ARCHITECTURE – NST NEWS CRAWLER PIPELINE

High Performance Web Crawling, Data Processing and Optimization



Data Collection

01

TARGET WEBSITE

New Straits
Times (NST)

02

METADATA COLLECTED

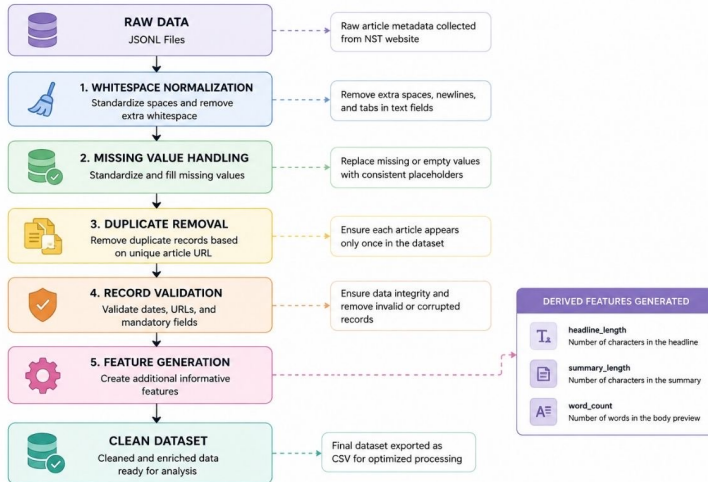
- Headline
- Article URL
- Publication Date
- Modified Date
- Author
- Section
- Summary
- Keywords
- Body Preview
- Word Count
- Collection Timestamp

03

RESULT

✓ Successfully
collected
102,265 news
article records

Data Processing



Optimization Techniques

01



MULTITHREADED WEB CRAWLING

- ThreadPoolExecutor
- Crawl multiple pages simultaneously
- Faster downloading

02



PARALLEL DATA CLEANING

- Parallel data cleaning
- Multiple CPU cores
- Faster preprocessing

03

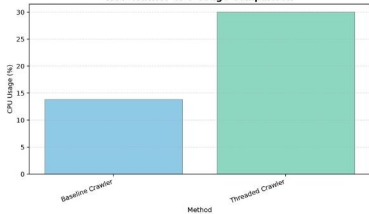


POLARS LAZY EXECUTION

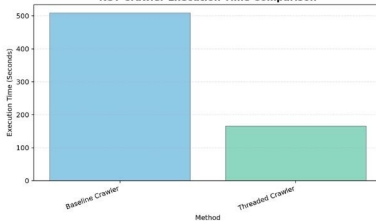
- LazyFrame
- Query optimization
- Automatic parallel execution

Performance Evaluation - Web Crawling

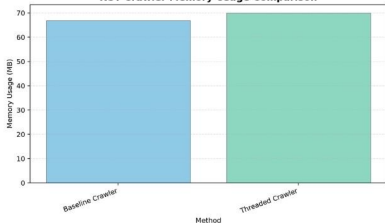
NST Crawler CPU Usage Comparison



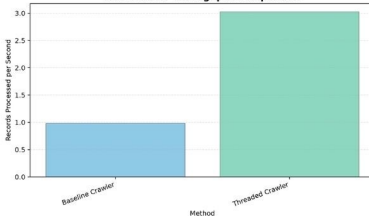
NST Crawler Execution Time Comparison



NST Crawler Memory Usage Comparison



NST Crawler Throughput Comparison

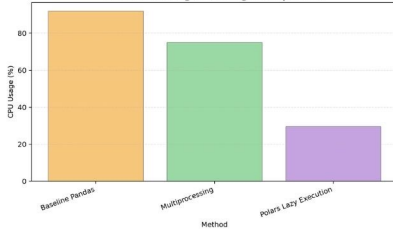


Performance Summary

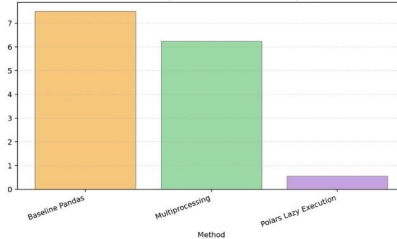
- Multithreading significantly reduced crawling execution time.
- Throughput increased significantly while maintaining data quality.
- System resources were utilized more efficiently than the baseline crawler.

Performance Evaluation - Data Processing

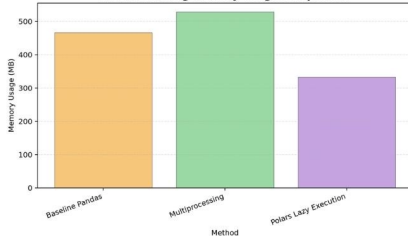
NST Processing CPU Usage Comparison



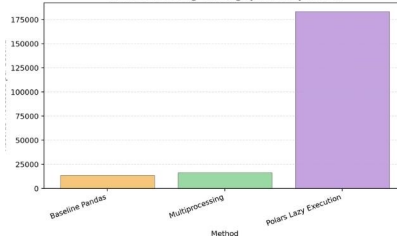
NST Processing Execution Time Comparison



NST Processing Memory Usage Comparison



NST Processing Throughput Comparison



Performance Summary

- Multiprocessing improved the efficiency of data cleaning.
- Polars Lazy Execution achieved the shortest execution time and the highest throughput.
- Polars provided the best overall processing performance with lower CPU and memory usage than the baseline.

Challenges & Limitations

Challenges Encountered



Network Dependency

- Crawling performance depends on internet speed and NST server response time.
- Execution time may vary between benchmark runs.



Balancing Speed and Ethics

- Too many threads can overload the target website.
- Number of threads and request delays were carefully configured.



Multiprocessing Overhead

- Creating and managing multiple processes consumes additional system resources.
- Process synchronization is required before merging results.

Limitations



Website-Specific Evaluation

- Performance results are based only on the NST website.
- Results may differ for websites with different structures or dynamic content.



Workload Dependency

- Polars performs best for the current dataset.
- Different datasets or analytical tasks may produce different results.



Hardware Dependency

- Performance depends on CPU, memory, operating system, and network conditions.
- Results may vary across different computing environments.

Conclusion & Future Work

Conclusion

- ✓ Built an automated NST web crawler
- ✓ Collected 102,265 records
- ✓ Successfully implemented
 - Multithreading
 - Multiprocessing
 - Polars Lazy Execution
- ✓ Performance improved significantly

Future Work

- 🚀 Asynchronous Crawling
 - AsyncIO / Async HTTP libraries
- 🌐 Multiple News Sources
 - Support multiple Malaysian news website
- ⚡ Distributed Processing
 - Apache Spark or Dask
 - Handle millions of records
- 📊 Advanced News Analytics
 - Topic Classification
 - Sentiment Analysis
 - Keyword Extraction
 - Trend Analysis



